

Digital Systems to Modern Computer Architecture

RISC-V ARCHITECTURE SCHOOL · Session 1

Prof. Dr. Tassadaq Hussain

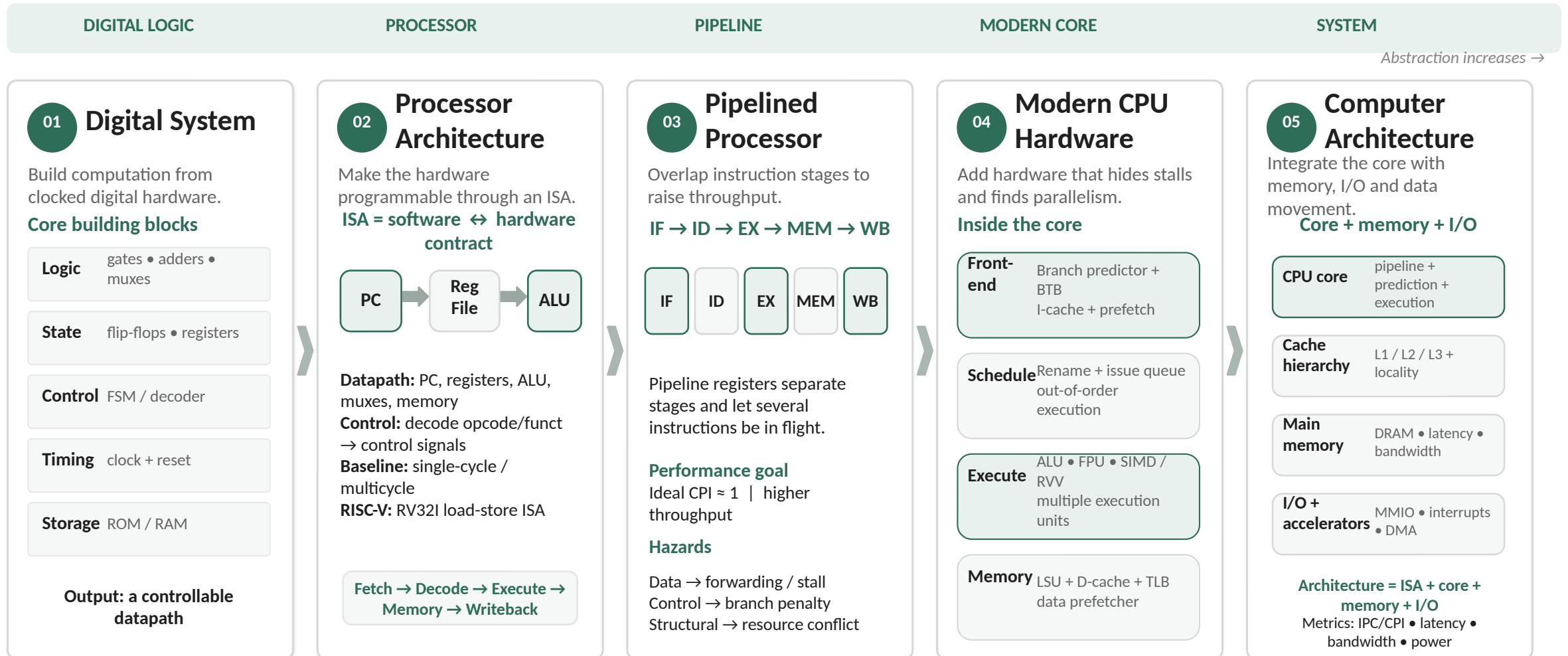
Processor fundamentals · RISC-V ISA · pipelining · modern CPU hardware ·
memory and system architecture

How the Session Fits Together

01	Digital system	Logic + state
02	Processor	Datapath + control
03	ISA	Software contract
04	Pipeline	Overlap + hazards
05	Modern core	Predict + schedule
06	System	Memory + I/O

From Digital Systems to Modern Computer Architecture

A coherent learning path: circuits → processor datapath → pipeline → modern CPU hardware → complete computer system



Part 1

Digital Systems

Start with the hardware primitives that make computation possible: logic, state, timing, datapath and control.

What Is a Digital System?

LOGIC + STATE

A clocked hardware system transforms binary inputs and stored state into useful outputs.

Inputs

Binary values enter from sensors, memory, peripherals or other logic.

Combinational logic

Gates, adders, comparators and multiplexers compute from current inputs.

State / registers

Flip-flops and registers preserve values from one clock cycle to the next.

Outputs

The transformed values drive memory, interfaces or the next hardware stage.

Combinational vs Sequential Logic

The defining difference is whether the circuit remembers anything.

Combinational

Output depends only on current inputs

Examples: ALU, decoder, mux, comparator

No stored history

Propagation delay limits timing

Sequential

Output depends on inputs plus stored state

Examples: registers, counters and FSMs

State updates at clock edges

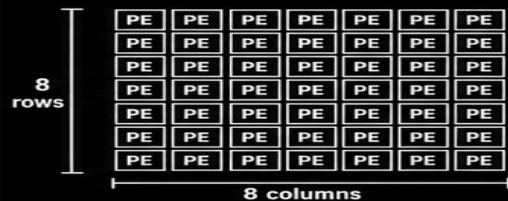
Enables instruction-by-instruction execution

SAME 8×8 MATMUL — DIFFERENT HARDWARE, DIFFERENT CYCLES

$$A \begin{matrix} \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \end{matrix} \times B \begin{matrix} \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \end{matrix} = C \begin{matrix} \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} & \boxed{} \end{matrix}$$

64 DOT PRODUCTS | EACH: 8 MUL + 7 ADD

64 DOT-PRODUCT PEs

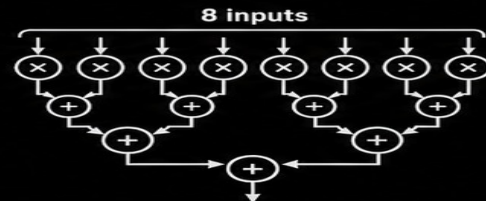


64 outputs at once

HARDWARE: 512 MUL + 448 ADD

1 CYCLE

ONE 8-LANE DOT UNIT



1 output each cycle $\times$ 64

HARDWARE: 8 MUL + 7 ADD

64 CYCLES

ONE MAC UNIT

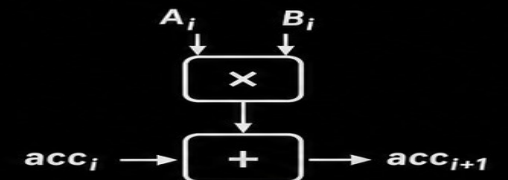


8 cycles/output $\times$ 64

HARDWARE: 1 MAC

512 CYCLES

SCALAR MUL + ADD



15 cycles/output $\times$ 64

HARDWARE: 1 MUL + 1 ADD

960 CYCLES

◀ MORE PARALLELISM • MORE AREA/POWER • FEWER CYCLES

MORE REUSE • LESS AREA/POWER • MORE CYCLES ▶

IDEAL COMPUTE-ONLY MODEL: operands ready; excludes memory, control, stalls, and pipeline fill. Execution cycles $\neq$ CPU CPI.

Clock, State and Register Transfer

Current cycle

Between edges

Next cycle

Register A

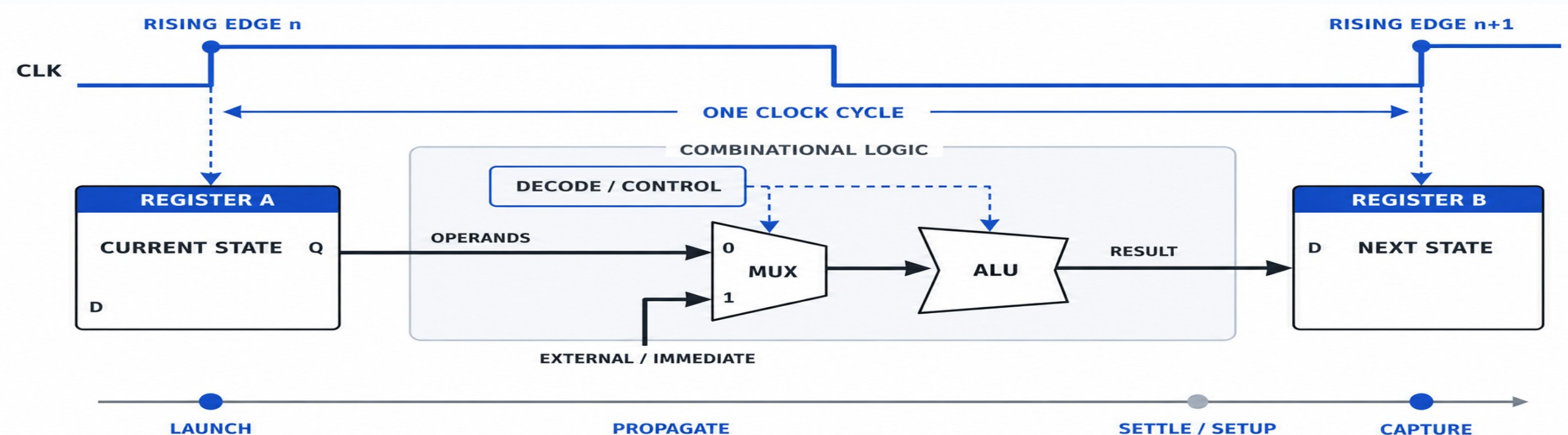
Holds the current state at the active clock edge.

Combinational logic

ALU, mux and decode logic transform the state.

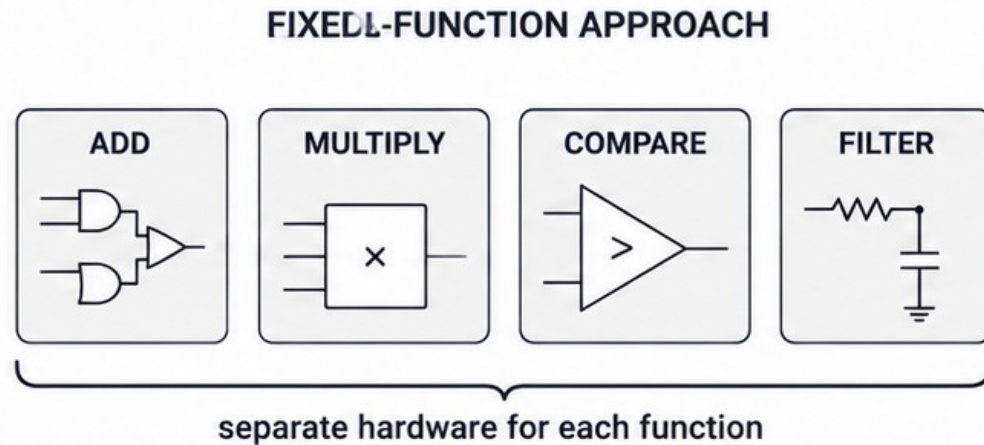
Register B

Captures the next state when the following edge arrives.

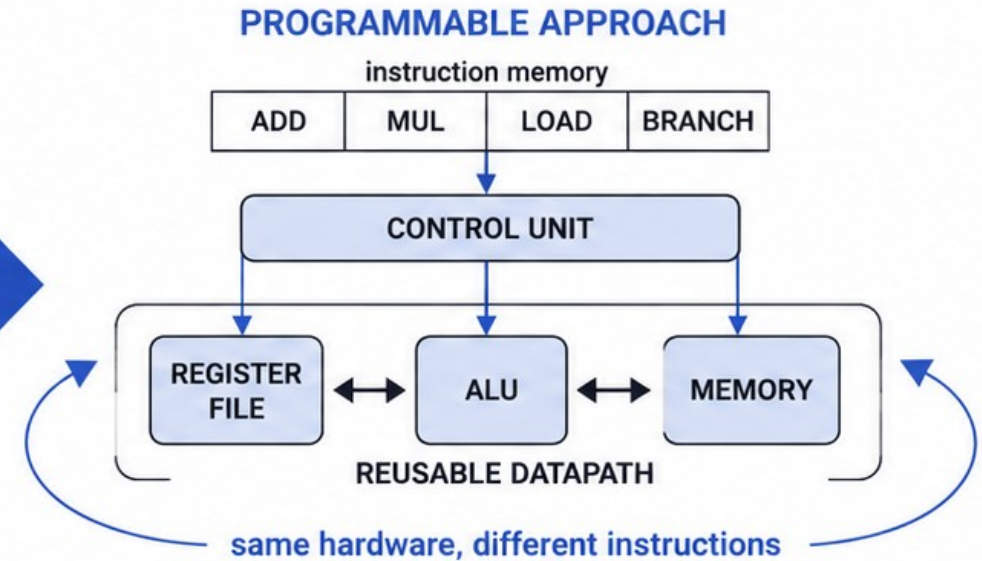


Why Build a Programmable Processor?

One reusable datapath can perform many functions—the program changes, not the silicon.



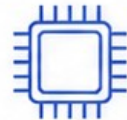
New function → new circuit



Many functions → one reusable system



REPROGRAMMABLE



HARDWARE REUSE



LESS DUPLICATED LOGIC



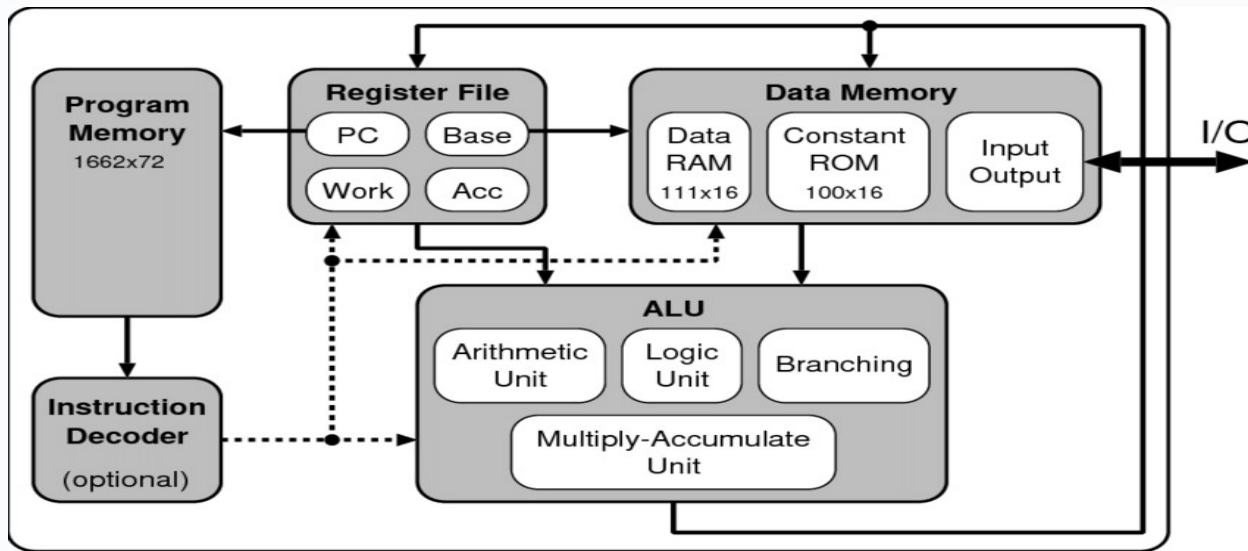
LOWER TOTAL AREA + IDLE POWER*

*For a system supporting many functions. Dedicated hardware can still be faster and more energy-efficient for one fixed task.

Datapath and Control

Most processors can be understood as two cooperating structures.

- ❑ A **datapath** contains all the functional units and connections necessary to implement an instruction set architecture
 - PC, Instruction ROM, Register File, ALU, Adders/MUXes
- ❑ The **control unit** tells the datapath what to do, based on the instruction that's currently being executed
 - RegWrite, ALUSel, PCWrite (and later MemRead/MemWrite)
 - The control signals can be generated by a combinational circuit with the instruction's opcode and func fields as the input



Control decoder

Interprets opcode, funct3 and funct7.

Steering signals

Generates RegWrite, ALUSrc, MemWrite and MemToReg.

Next-PC control

Uses branch conditions and PCSrc to choose the next instruction. 9

From Information to Computation

STORED PROGRAM

Programs and data live in memory; the processor repeatedly fetches and transforms them.



Program memory

Stores encoded instructions: Inst 1, Inst 2, Inst 3 and beyond.



Processor

Fetches, decodes, executes and writes results back.



Data memory

Supplies operands and stores Data 1, Data 2, Data 3 and beyond.

The Stored-Program Computer

SYSTEM VIEW

The classic model connects processor, memory, storage and I/O through address, data and control buses.



CPU

Datapath and control execute the stored program.



Main memory

Holds active program instructions and data.



Storage + I/O

Retains persistent data and connects peripherals to the system.

Part 2

Processor Architecture

Make the digital system programmable by defining an ISA and implementing it with datapath and control.

Processor Architecture Has Three Levels

KEEP THE LEVELS SEPARATE

Each level answers a different question about the same computer.



ISA

What software can ask the processor to do: instructions, registers and memory model.



Microarchitecture

How hardware implements the ISA: datapath, control, pipeline and prediction.



System architecture

How the processor connects to cache, DRAM, storage, DMA and I/O.

Core Processor Components

01 **Registers**

Fast local state

02 **ALU**

Arithmetic + logic

03 **Decoder**

Instruction meaning

04 **Control unit**

Control signals

05 **Buses / muxes**

Data movement

06 **Clock**

Synchronization

Registers: The Processor's Fast State

Registers are a type of computer memory built directly into the processor that is used to store and manipulate data during the execution of instructions.

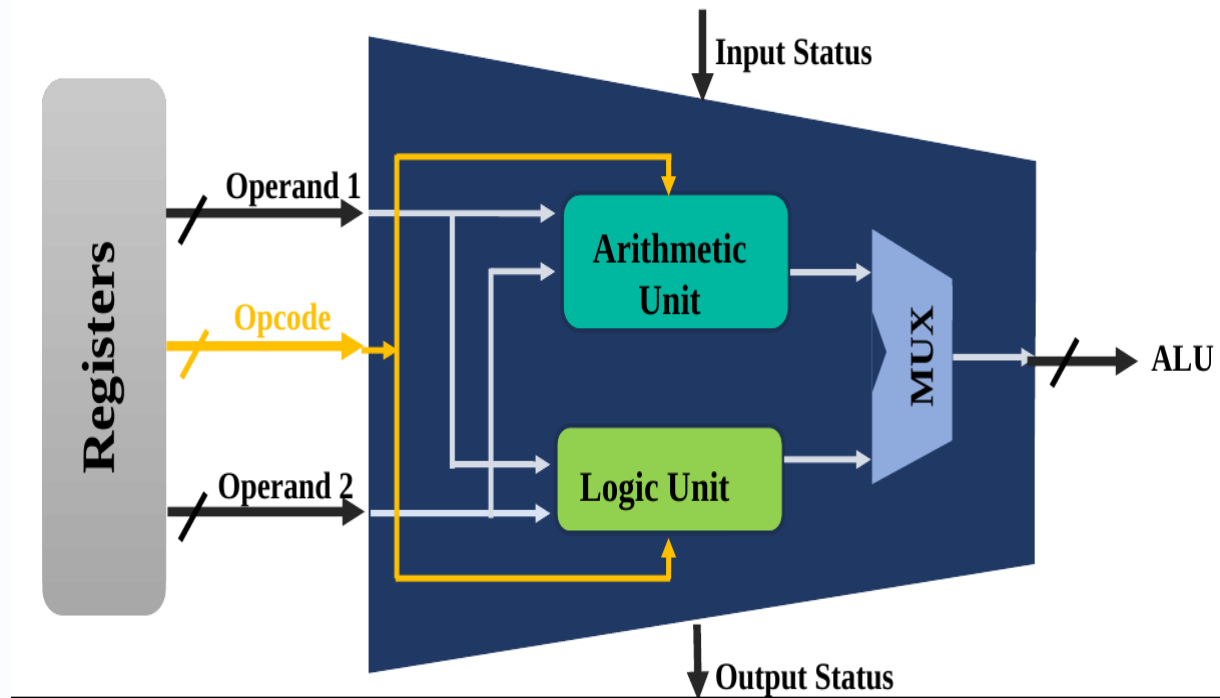
A register may hold an instruction, a storage address, or any kind of data (such as a bit sequence or individual characters).

01	PC	Next instruction address
02	IR	Current instruction; implementation-dependent in modern pipelines
03	Status / CSR	Control and architectural state
04	GPRs	Operands and results
05	SP	Top of stack
06	Address / data	Temporary microarchitectural state

Arithmetic Logic Unit (ALU)

INTEGER EXECUTION

The ALU is the main integer execution block in a simple processor.



Operand inputs

Operand A and Operand B come from registers, the PC or an immediate.



Operations

ADD, SUB, AND, OR, XOR, shifts and comparisons.



Outputs

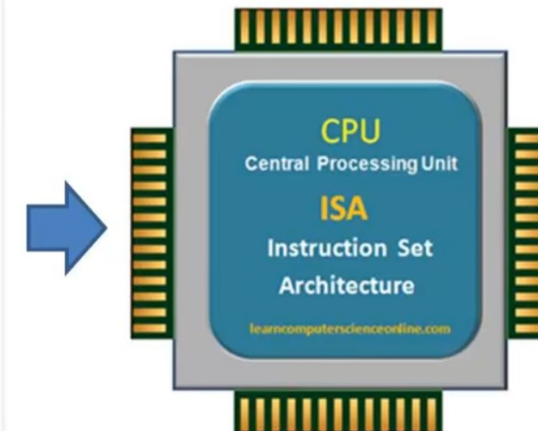
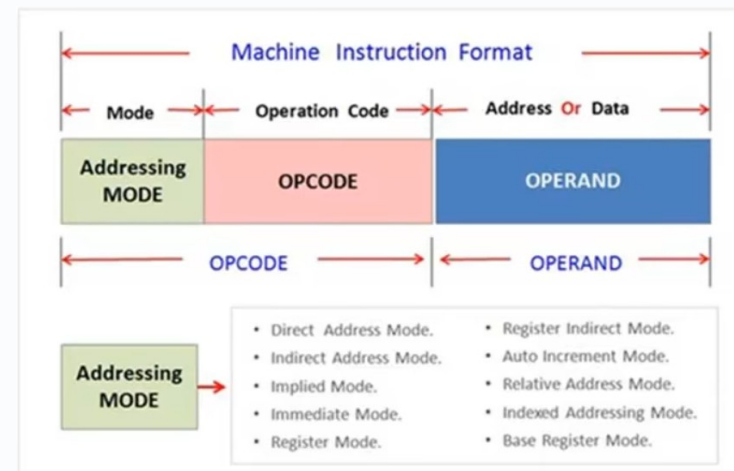
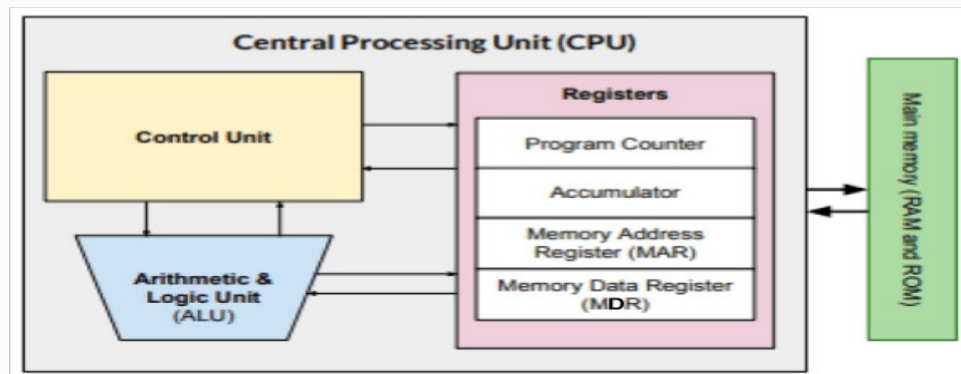
A result value plus condition information for control decisions.

Basic Processor Architecture

Processor Architecture refers to the design and organization of a processor's central processing unit (CPU).

Components of Processor:

- **Arithmetic and Logic Unit:** Performs mathematical calculations.
- **Control Unit:** Control the overall processing of the processor.
- **Decoders Unit:** Convert coded instructions into signals that can control other components.
- **Registers:** Hold data, instructions, and addresses temporarily during processing.
- **Buses:** Electrical pathways that transmit data and signals between components. Types include the data bus, address bus, and control bus.



$$\text{Execution Time} = \text{IC} \times \text{CPI} \times \text{Clock Period}$$

Program work

ONE EQUATION

Instruction Count (IC) is the number of instructions the program actually executes.

Design implication

Architecture shapes IC and CPI; circuit technology and pipeline balance shape the clock period.

Microarchitecture

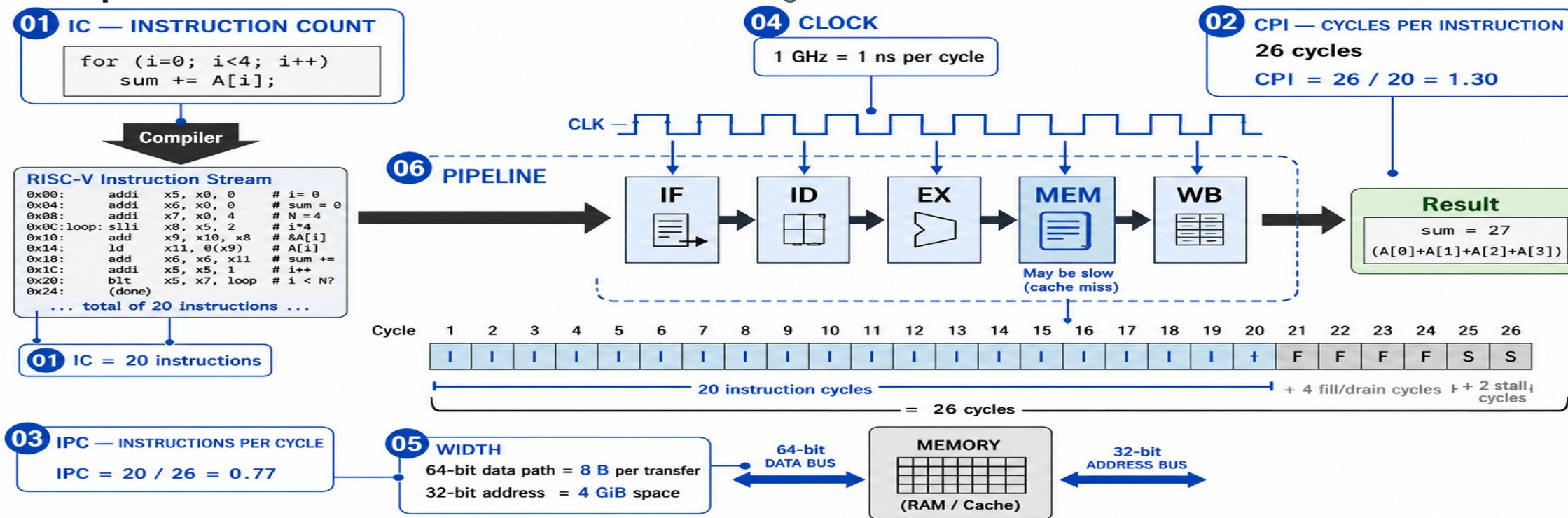
Cycles per Instruction (CPI) captures stalls, overlap, hazards and execution resources.

Circuit timing

Clock Period is the duration of each hardware cycle and is strongly influenced by pipeline depth and technology.

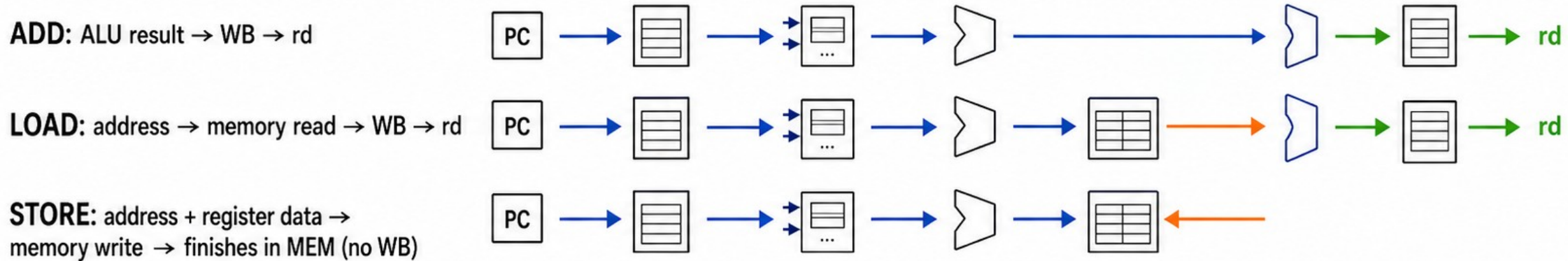
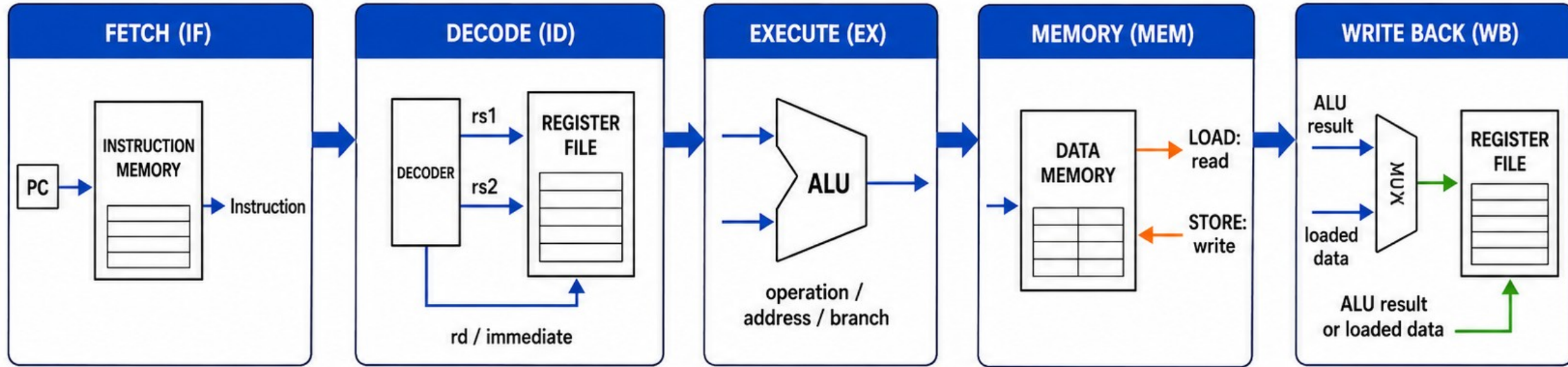
Clock Frequency Alone Does Not Determine Performance

- 01 IC Instruction count
- 02 CPI Cycles per instruction
- 03 IPC Instructions per cycle
- 04 Clock Cycle time / frequency
- 05 Width Data + address width
- 06 Pipeline Number and balance of stages



$$\text{Execution time} = IC \times CPI \times \text{cycle time} = 20 \times 1.30 \times 1 \text{ ns} = 26 \text{ ns}$$

FIVE STAGES OF INSTRUCTION EXECUTION



WB updates the register file; STORE writes memory in MEM.

Single-Cycle, Multicycle and Pipelined

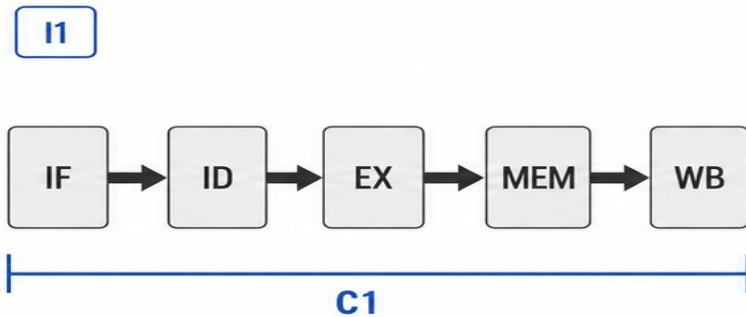
SAME ISA · DIFFERENT IMPLEMENTATIONS

Three implementation styles can execute the same instruction-set contract.

Single-cycle

One long cycle per instruction; simple control but a slow clock.

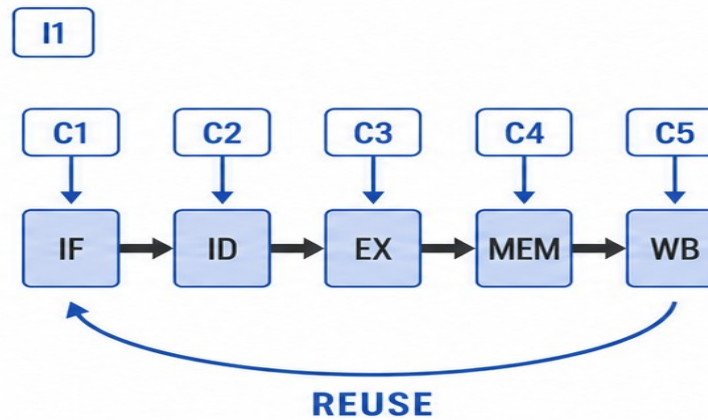
SINGLE-CYCLE



Multicycle

Split instructions into steps; reuse hardware with variable cycle counts.

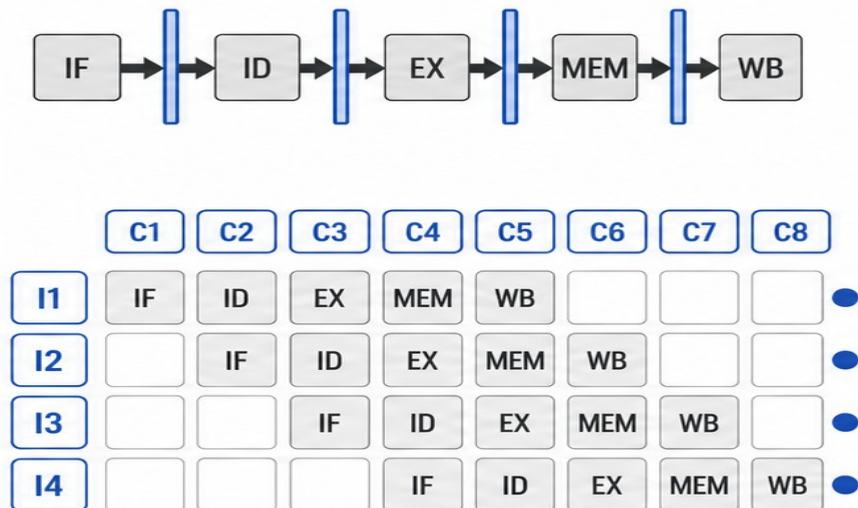
MULTICYCLE



Pipelined

Overlap stages for high throughput; manage hazards and control complexity.

PIPELINED



Part 3

Instruction Set Architecture

Define the programmer-visible contract and use RISC-V as the concrete example.

What the ISA Defines

Everything software must know to generate correct machine code.

Programmer-visible state

- Instruction vocabulary
- Register set and register width
- Instruction formats and encodings
- Addressing and memory-access rules
- I/O model exposed by the execution environment

Architectural behavior

- Data types and operations
- Control-flow semantics
- Exceptions and privileged state where applicable
- Observable results of every instruction
- Rules that remain stable across implementations

RISC vs CISC

Two instruction-set philosophies often converge internally on pipelined execution machinery.

RISC

- Simple, regular instructions

- Often fixed-width encoding

- Load/store organization is common

- Compiler-friendly scheduling

- Examples: RISC-V, ARM and MIPS

CISC

- Richer instruction forms

- Variable-length encodings are common

- More work can be expressed per instruction

- Complex decode or translation is often used

- Example: x86

Why RISC-V?

01 Open ISA

02 Royalty-free

03 Modular

04 Customizable

05 Toolchains

06 Scalable

Implementation freedom

Lower licensing barrier

Base + extensions

Domain-specific instructions

GCC + LLVM support

Embedded to high performance

RV32I Base ISA

32-BIT LOAD–STORE ARCHITECTURE

Only load/store instructions access memory; arithmetic operates on registers.

Arithmetic / logic

ADD · SUB · AND · OR · XOR · ADDI

Memory

LW reads memory; SW writes memory.

Control

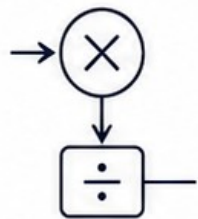
BEQ · BNE · JAL · JALR

Upper immediate

LUI and AUIPC build constants and PC-relative addresses.

RISC-V Extensions Through Simple C Examples

C expresses the operation; the compiler selects an enabled ISA extension.

**M**

M — Multiply / Divide

```
int p = a * b; // may use MUL
int q = a / b; // may use DIV
```

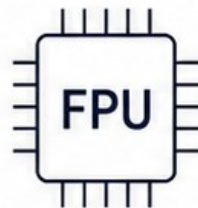
- Integer multiplication and division in hardware.

**A**

A — Atomics

```
old = atomic_fetch_add(&count, 1);
// indivisible read-modify-write; may use AMOADD
```

- Safe shared-data update between cores or threads.

**F/D**

F / D — Floating Point

```
float s = x + y; // F: may use FADD.S
double r = a / b; // D: may use FDIV.D
```

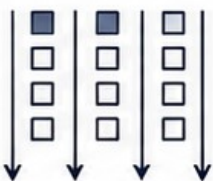
- F is 32-bit; D is 64-bit floating point.

**C**

C — Compressed

```
x += 1; // may encode as 16-bit C.ADDI
```

- Same operation, smaller instruction encoding.
- Chosen by the compiler; no special C syntax.

**V**

V — Vector Processing

```
for (int i=0; i<n; i++) c[i]=a[i]+b[i];
// may vectorize: many elements per instruction
```

- Data-parallel processing of arrays.

**B/Z***

B / Z* — Bit Manipulation

```
ones = __builtin_popcount(x);
// Zbb may use CPOP to count set bits
```

- Focused Zba/Zbb/Zbs extensions accelerate bit operations.



Important: `a = a + a;` is **NOT** atomic. Use C11 atomic operations for shared data.

Instruction Format: The Core Fields

- 01

funct7 / imm

7 bits; operation detail or immediate
- 02

rs2

5-bit source register
- 03

rs1

5-bit source register
- 04

funct3

3-bit operation selector
- 05

rd / imm

5-bit destination or immediate field
- 06

opcode

7-bit instruction class

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0		
funct7				rs2			rs1		funct3		rd			opcode		R-type
imm[11:0]						rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type
imm[12]	imm[10:5]			rs2			rs1		funct3		imm[4:1]	imm[11]	opcode		B-type	
imm[31:12]										rd			opcode		U-type	
imm[20]	imm[10:1]				imm[11]		imm[19:12]				rd			opcode		J-type

R-Type: Register-to-Register Operation

EXAMPLE · add x3, x1, x2

Semantic form: rd = rs1 (op) rs2.

Read

Read source registers x1 and x2.

Execute

Select the ALU operation using
opcode, funct3 and funct7.

Write

Write the ALU result into destination
register x3.

I-Type: Register + Immediate

EXAMPLE · `addi x3, x1, 16`

Semantic form: `rd = rs1 (op) imm12.`



Read

Read source register x1.



Prepare + execute

Sign-extend the 12-bit immediate and perform the selected ALU operation.



Write

Write the result into destination register x3.

S-Type: Store to Memory

EXAMPLE · `sw x2, 0(x1)`

Semantic form: $\text{mem}[\text{rs1} + \text{offset}] = \text{rs2}$.



Read operands

Read base address x1 and store data x2.



Calculate address

Reconstruct the split immediate and compute $\text{x1} + \text{offset}$.



Write memory

Assert MemWrite and store the x2 value at the effective address.

B-Type: Conditional Branch

EXAMPLE · `beq x1, x2, label`

Compare the operands, compute the target and select the next PC.



Compare

Test rs1 against rs2 using the branch condition.



Compute target

Add the decoded branch offset to the current PC.



Select next PC

Choose the target when taken; otherwise use PC + 4.

U- and J-Type: Large Immediates and Jumps

These formats support upper-immediate construction and long PC-relative control transfers.

U-Type

LUI loads a 20-bit upper immediate

AUIPC adds an upper immediate to the PC

Useful for building addresses and constants

Fields: imm[31:12] + rd + opcode

J-Type

JAL performs an unconditional PC-relative jump

rd receives the return address

Immediate bits encode the jump target offset

Forms the basis of calls with a link register

Load-Store Architecture

REGISTER OPERATIONS · MEMORY TRANSFERS

Arithmetic instructions do not directly operate on memory operands in RV32I.



Registers

x0...x31 hold operands and results for computation.



ALU

Performs register operations and calculates effective addresses.



Data memory

LW and SW are the only base instructions that access data memory.

Part 4

Single-Cycle RISC-V Core

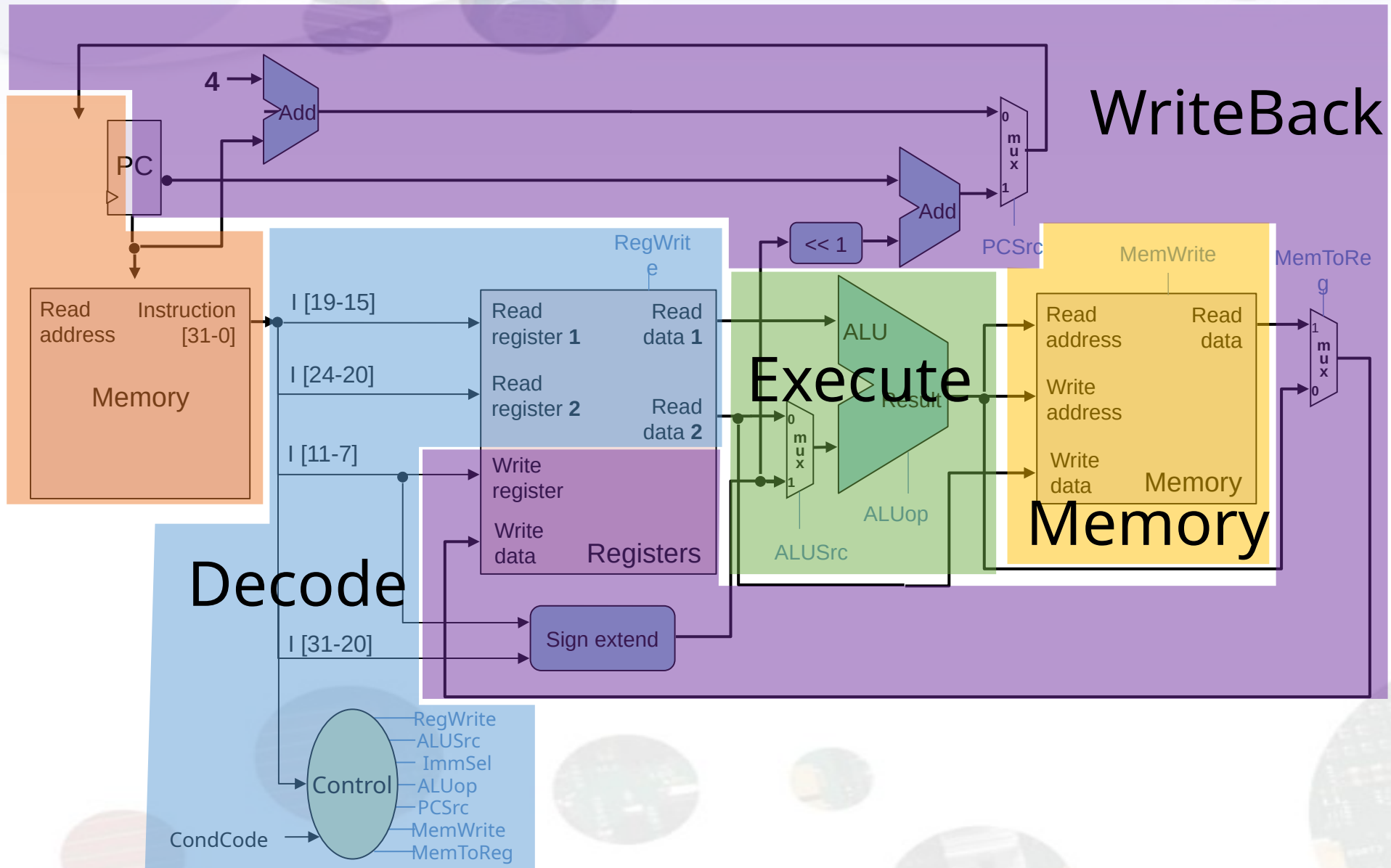
Map the ISA onto hardware before introducing pipelining.

Five Conceptual Stages Apply to Every Implementation

01	Fetch	Read instruction at the PC
02	Decode	Read operands and generate control
03	Execute	ALU operation or branch comparison
04	Memory	Access data for loads and stores
05	Write back	Update the destination register
06	Next PC	PC + 4 or an alternate branch / jump target

One Instruction Traverses the Required Blocks in One Cycle

01	PC	Select instruction address
02	Instruction memory	Return instruction bits
03	Register file	Read operands
04	ALU	Compute result or address
05	Data memory	Serve load / store
06	Writeback + control	Select result and update PC / rd



Stage 1 — Instruction Fetch (IF)

READ THE NEXT INSTRUCTION

The program counter supplies the address and advances toward the next sequential instruction.



PC

Holds the address of the current instruction.



Instruction memory

Returns instruction bits [31:0] for decode.



PC + 4

Computes the sequential next address before branch or jump selection.

Stage 2 — Decode and Register Read

INTERPRET + PREPARE

Decode instruction fields, read operands and reconstruct the immediate.

Instruction fields

Opcode, rs1, rs2, rd, funct3, funct7 and immediate bits.

Register file

Read rs1 and rs2; identify destination rd.

Immediate generator

Reconstruct and sign-extend I, S, B, U or J immediates.

Control decoder

Generate RegWrite, ALUSrc, MemWrite, PCSrc, ImmSel and MemToReg.

Stage 3 — Execute (EX)

ALU WORK

The ALU performs arithmetic, logic, comparison or effective-address calculation.



Operand 1

Select rs1 or the PC.



Operand 2

Select rs2 or the decoded immediate.



ALU result

Produce an arithmetic value, condition or memory address.

Stage 4 — Memory Access (MEM)

LOADS + STORES

ALU instructions normally bypass data memory; loads and stores use the effective address.



Effective address

The ALU result selects the target memory location.



Store path

rs2 data is written when MemWrite is asserted.



Load path

Memory returns data for the following writeback stage.

Stage 5 — Write Back (WB)

UPDATE rd

Writeback selects the architecturally correct result when RegWrite is asserted.



ALU result

Arithmetic, logic and effective-address results.



Memory data

The value returned by a load instruction.



WB mux + register file

Select ALU, load or PC + 4 and write destination rd.

lw x5, 0x222(x0) Uses the Full Datapath

- | | |
|---------------------|---|
| 01 IF | Fetch the encoded load instruction |
| 02 ID | Read x0 and sign-extend 0x222 |
| 03 EX | Calculate address $0 + 0x222$ |
| 04 MEM | Read the addressed data word |
| 05 WB | Write the loaded value into x5 |
| 06 Key point | A load determines the longest single-cycle path |

Part 5

Pipelined Processor

Overlap five stages to increase throughput, then manage the hazards created by that overlap.

Why Pipeline the Processor?

Break the long instruction path into shorter stages and overlap different instructions.

Single-cycle

- One instruction uses the full datapath
- The next instruction waits
- Clock period must fit the longest path
- Low overlap and limited throughput

Pipelined

- Different stages process different instructions
- Shorter stage delays enable a faster clock
- Ideal throughput approaches one completion per cycle
- Hazards introduce stalls and recovery work

State Must Travel Between the Five Stages

01	IF / ID	Instruction and PC
02	ID / EX	Operands, immediate and control
03	EX / MEM	ALU result, store data and control
04	MEM / WB	Memory / ALU result and writeback control
05	WB	Destination register update
06	Rule	Control signals travel with the instruction until used

After Filling, Multiple Instructions Occupy the Pipeline

- | | | |
|-----------|---------------------|---|
| 01 | Cycle 1 | I1 enters IF |
| 02 | Cycle 2 | I1: ID · I2: IF |
| 03 | Cycle 3 | I1: EX · I2: ID · I3: IF |
| 04 | Cycle 4 | I1: MEM · I2: EX · I3: ID · I4: IF |
| 05 | Cycle 5 | I1: WB while four younger instructions continue |
| 06 | Steady state | Ideally one instruction completes each cycle; $CPI \approx 1$ |

$$\text{CPI}_{\text{actual}} = \text{CPI}_{\text{ideal}} + \text{CPI}_{\text{data}} + \text{CPI}_{\text{control}} + \text{CPI}_{\text{memory}}$$

Ideal throughput

IDEAL ≈ 1

Once the pipeline is full, the ideal machine completes roughly one instruction per cycle.

Interpretation

Pipeline performance depends on how effectively the design prevents, hides or recovers from stalls.

Hazard cost

Data dependencies and control uncertainty insert bubbles or flush wrong-path work.

Memory cost

Cache and DRAM delays add further cycles when operands or instructions are unavailable.

Pipeline Hazards

WHY THE NEXT INSTRUCTION MAY WAIT

Overlap creates conditions where an instruction cannot safely proceed.



Data hazard

An operand is not ready. Forward the value or stall.



Control hazard

The next PC is uncertain. Predict and recover on a miss.



Structural hazard

A resource is busy. Duplicate the resource or stall.

Data Hazard: RAW Dependency

A new instruction needs a value that an older instruction has not written yet.

Example

Instruction 1: add x5, x1, x2

Instruction 2: sub x6, x5, x3

Instruction 2 consumes x5 immediately

Without help, the pipeline must wait

Producer

The add creates the new x5 value.

Consumer

The sub requires x5 as an input.

Dominant simple hazard

RAW is the key data hazard in a basic in-order five-stage pipeline.

Forwarding (Bypassing)

SEND THE RESULT DIRECTLY

Forward a value from a later pipeline stage to the dependent execution stage.

Source

Take an ALU result from EX/MEM or MEM/WB.

Forward path

Route the value directly to the EX-stage operand mux.

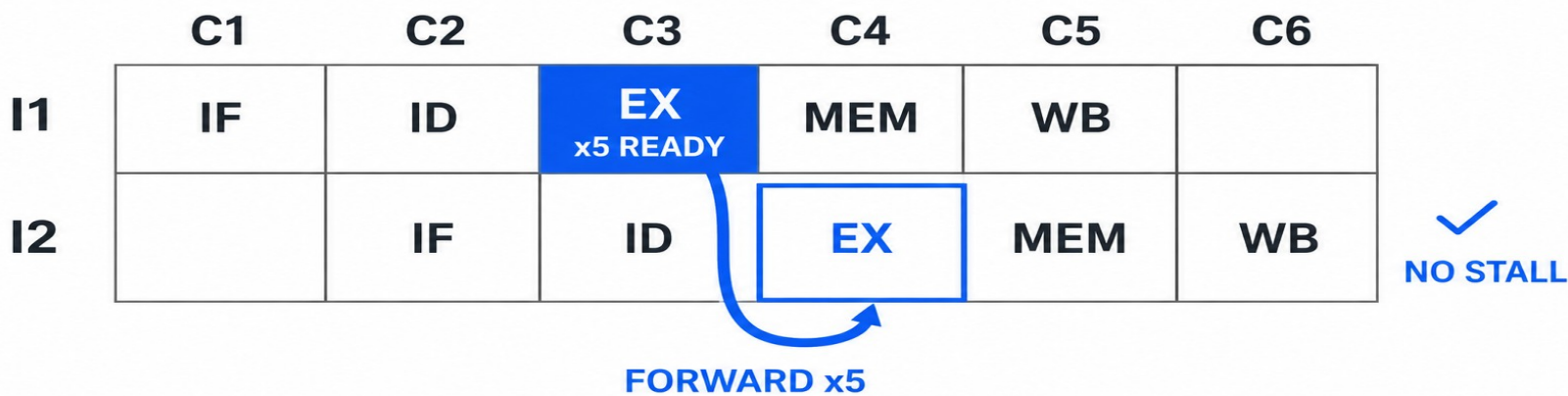
Benefit + limit

Avoid many ALU-to-ALU stalls, but only when the value is already available.

I1: ADD x5, x1, x2



I2: SUB x6, x5, x3



Why Forwarding Is Not Always Enough

A load-use dependency may still require a bubble because memory data arrives later.

Example

```
lw x5, 0(x1)
```

```
add x6, x5, x3
```

The dependent add reaches EX before load data is ready

Hazard detection inserts a stall / bubble

Load timing

The requested value arrives after the memory stage.

Hazard detection

Hold the dependent instruction for one cycle.

Then forward

Supply the value as soon as memory returns it.

Control Hazard: Branch Uncertainty

Fetch needs the next PC before the branch may be resolved.

Branch example

beq x1, x2, LABEL

Fall-through: add x3, x4, x5

Target: LABEL: mul ...

The machine must choose a path early

Wait

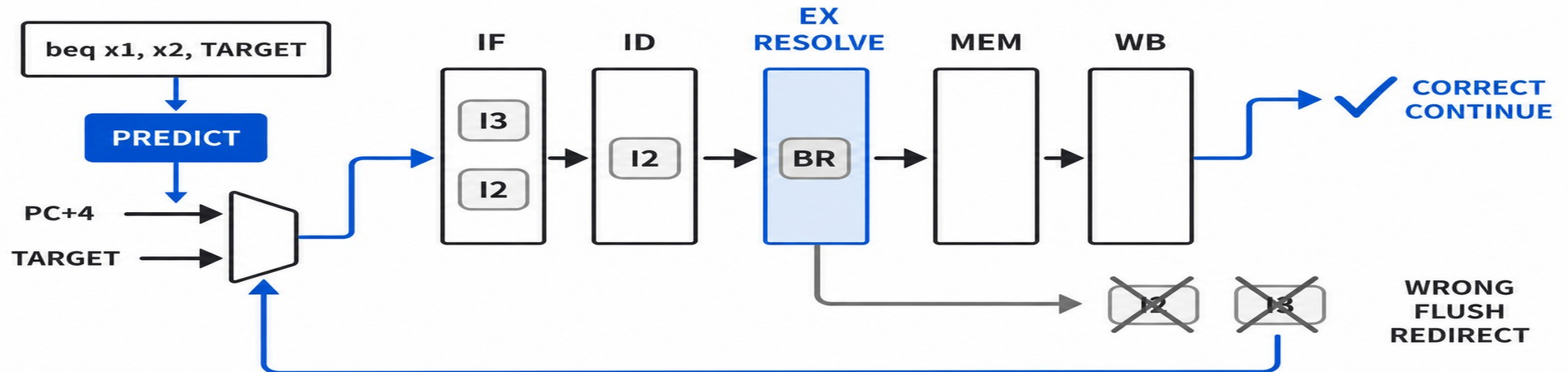
Stall fetch until the branch decision is known.

Predict

Continue on the most likely path.

Recover

Flush younger wrong-path instructions after a misprediction.



Part 6

Modern CPU Hardware

Add prediction, scheduling and memory-side structures to keep the pipeline busy.

Branch Prediction Unit

Predict taken or not-taken before the branch is fully executed.

Prediction path

Use the current PC as the lookup key

Combine local or global history

Predict a direction

Select a predicted next PC

Correct prediction

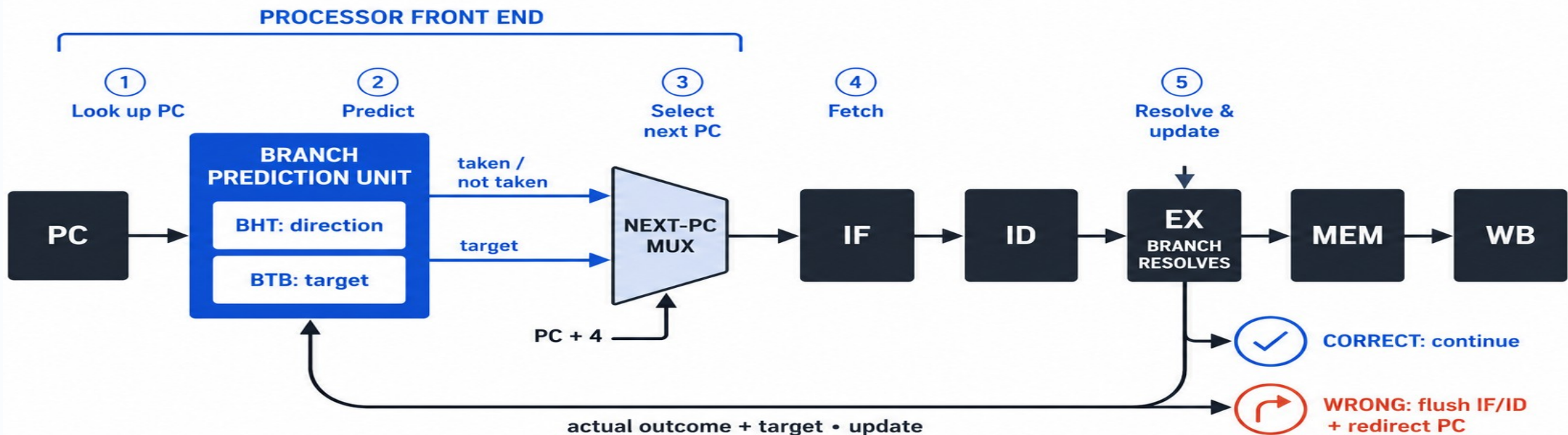
Fetch continues without waiting, reducing the control stall.

Wrong prediction

Wrong-path work is flushed and the pipeline pays a recovery penalty.

Design goal

Improve accuracy without delaying the front-end clock path.



Branch Target Buffer (BTB)

PREDICT WHERE TO GO

Direction prediction says whether to branch; the BTB supplies the predicted target address.



Branch PC

Lookup key identifying the control-flow instruction.



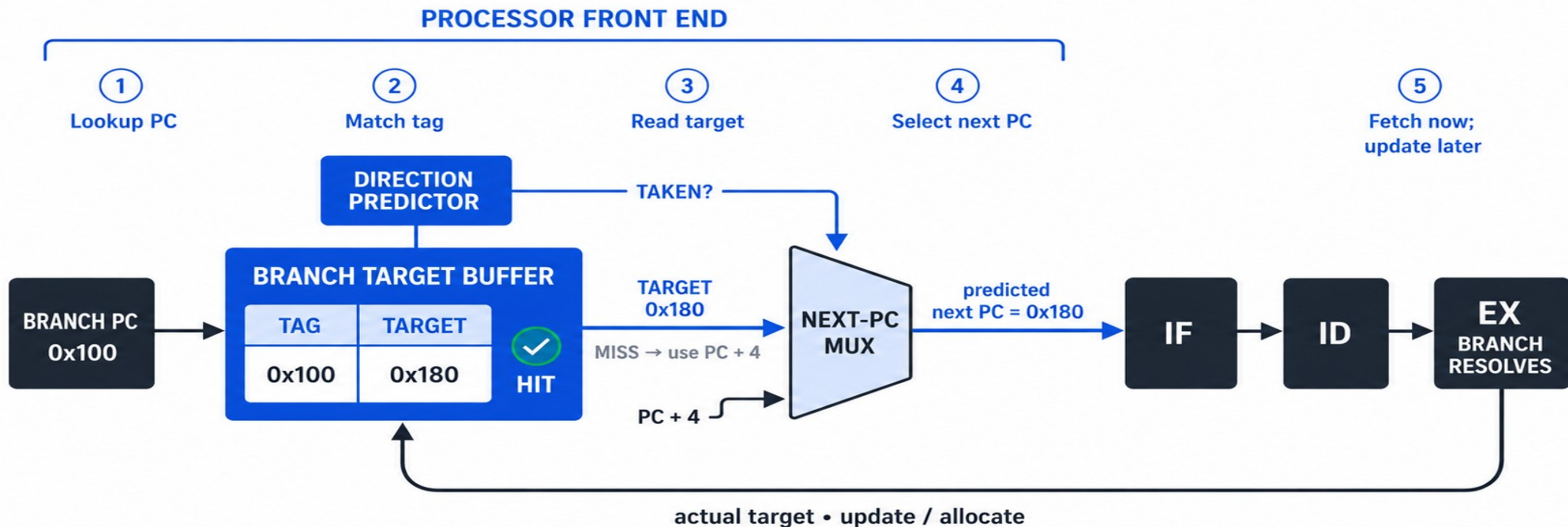
BTB entry

Tag = branch PC; value = previously observed target PC.



Target PC

Feeds the next fetch address for a predicted-taken branch.



Instruction Prefetcher

Bring likely future instruction-cache lines before the fetch unit demands them.

Front-end path

Observe the PC stream

Predict future instruction lines

Request them from lower memory

Place them in the I-cache or fetch queue

Works well

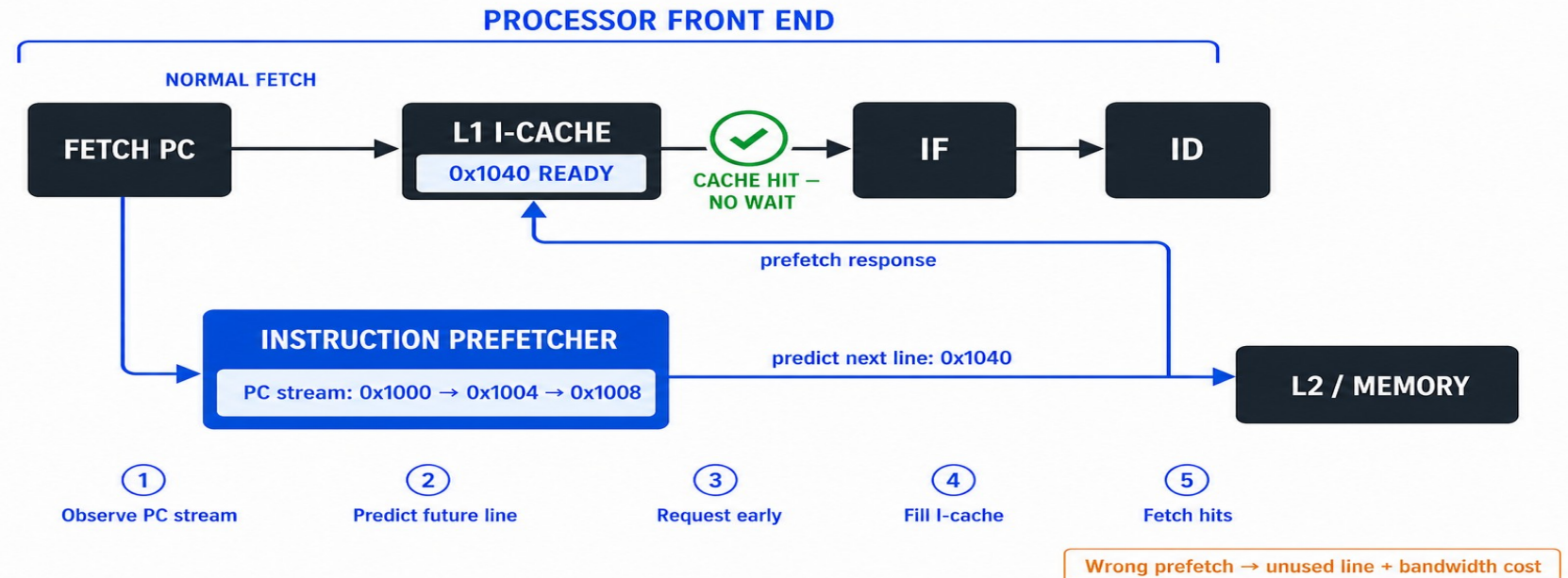
Sequential code, loops and predictable control flow.

Trade-off

Wrong prefetches waste bandwidth and can evict useful lines.

Timeliness

A correct prefetch helps only if it arrives before demand.



Instruction Fetch Must Continuously Feed the Back-End

- | | | |
|-----------|---------------------------|---|
| 01 | Next-PC select | Choose sequential, target or recovery address |
| 02 | Branch predict | Predict direction and target |
| 03 | I-cache + prefetch | Supply likely instruction lines |
| 04 | Fetch buffer | Absorb delivery variation |
| 05 | Decode | Translate instructions for execution |
| 06 | Outcome | Deliver a steady stream of useful work |

Superscalar Execution

MULTIPLE INSTRUCTIONS PER CYCLE

Issue independent instructions in parallel using multiple execution resources.

Decode width

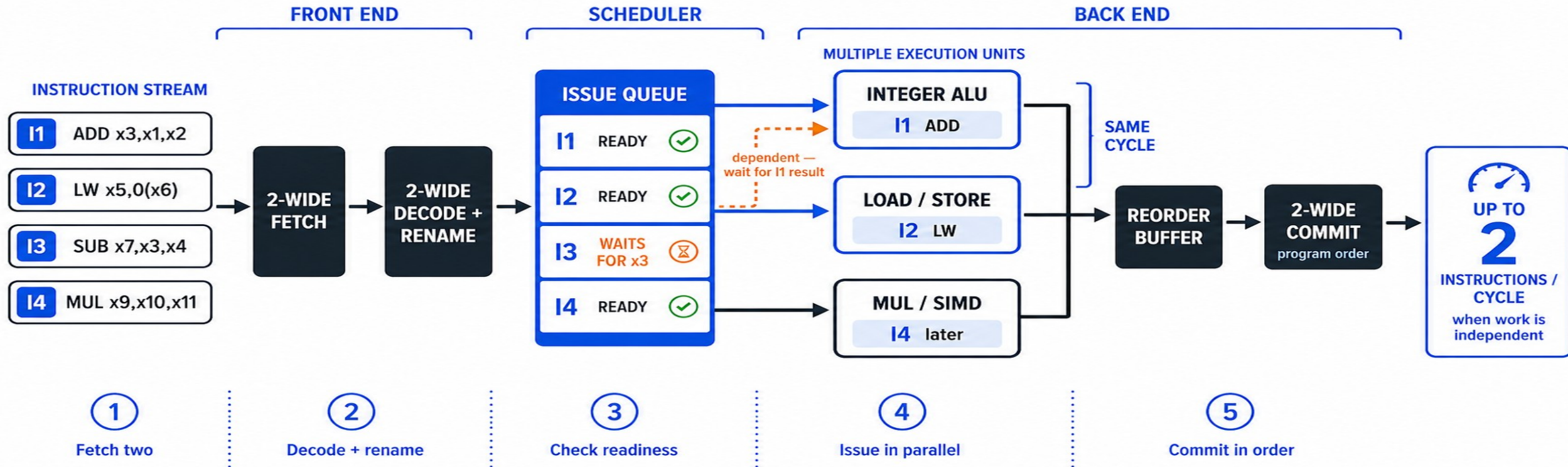
Decode and rename several instructions per cycle.

Execution resources

Multiple integer ALUs plus load/store, floating-point or SIMD units.

Higher IPC

Throughput rises when the instruction stream contains enough independent work.



Out-of-Order Execution

Run independent instructions when ready, even if an older instruction is stalled.

Core flow

Rename removes false dependencies

Issue queue tracks operand readiness

Execution units run available work

ROB holds in-flight results

Commit preserves architectural order

Why it helps

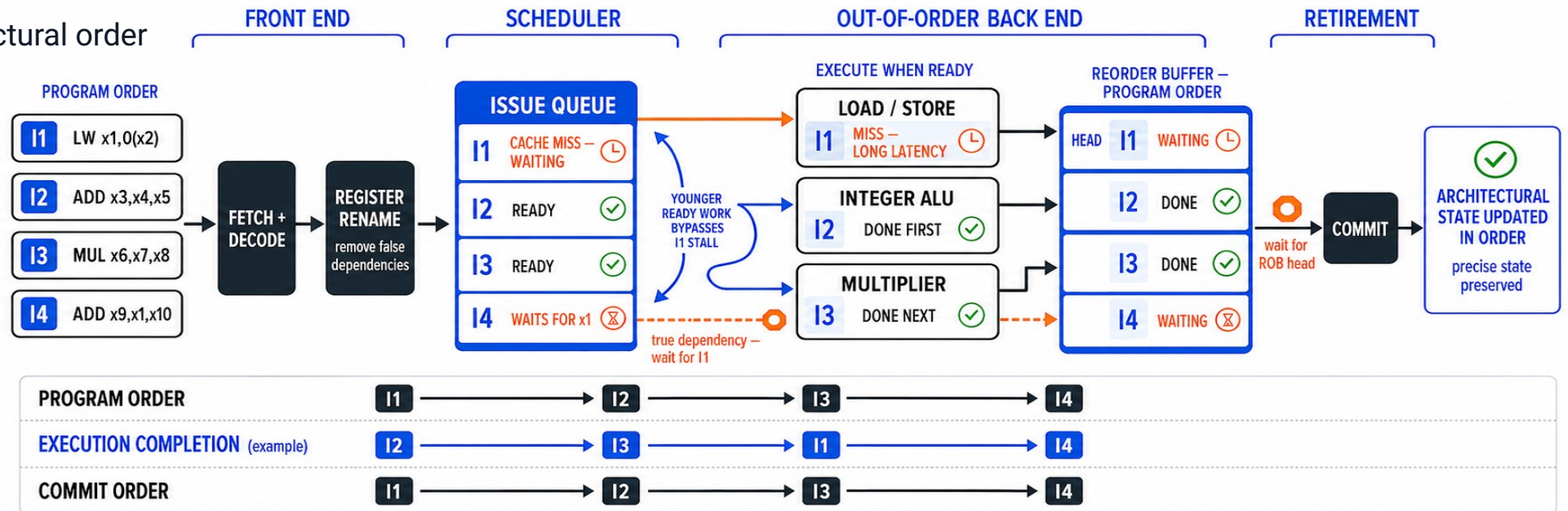
Hides memory and long-operation latency with independent work.

Execution order

Instructions may finish out of program order.

Correctness rule

Architectural retirement remains in order.



- 1 Fetch in order
- 2 Rename
- 3 Track readiness
- 4 Execute ready work
- 5 Hold results in ROB
- 6 Commit in order

Register Renaming

REMOVE FALSE DEPENDENCIES

Map architectural registers to a larger physical register file.



Architectural registers

Software-visible names such as x5, x6 and x7.



Rename table

Maps each architectural name to its current physical register.



Physical registers

A larger pool such as p17, p22 and p31 holds distinct versions.

Issue Queue / Reservation Station

WAIT UNTIL READY

Decoded and renamed instructions wait until operands and execution units are available.

ADD p17,p3,p4

Operands ready — eligible to issue.

MUL p20,p8,p9

Operands ready but waiting for a multiplier.

ADD p22,p17,p5

Waiting for the producer of p17.

LOAD p30,[p2]

Ready when the load/store path is free.

Reorder Buffer and Commit

PRECISE ARCHITECTURAL STATE

Commit waits for the oldest safe instruction, even when younger work has finished.

I1 · done

Oldest completed instruction can commit.

I2 · waiting

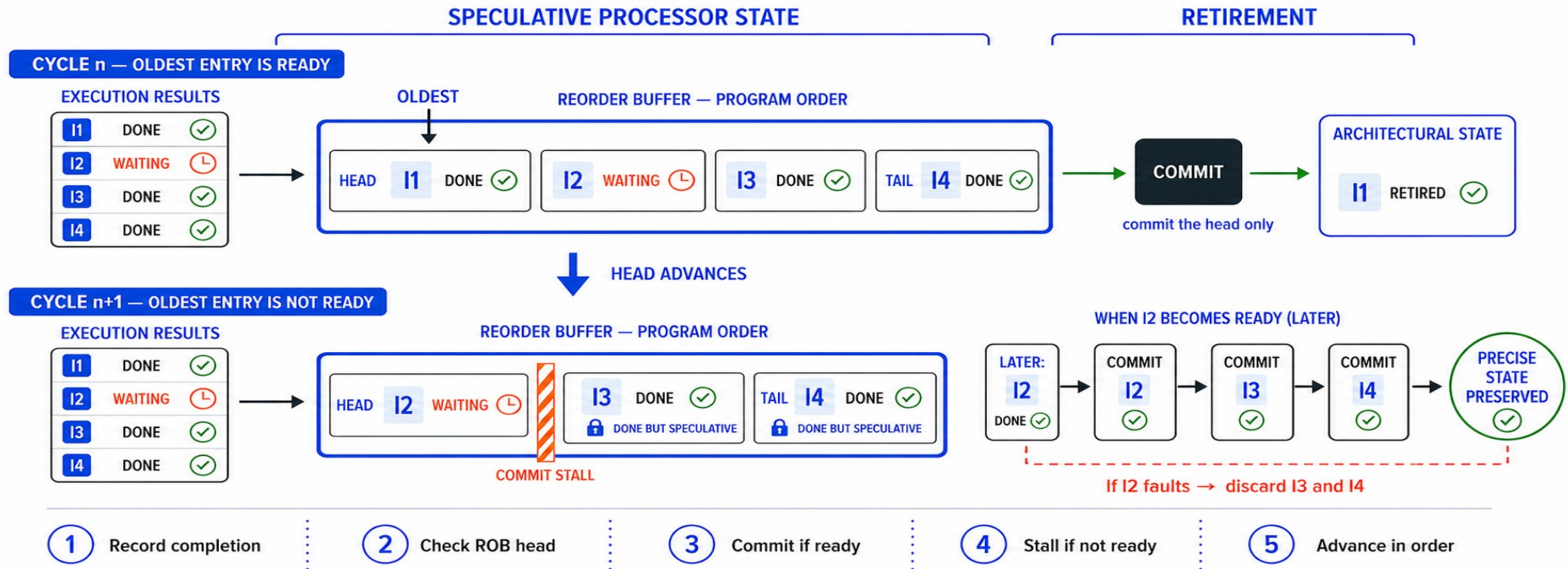
Blocks younger instructions from retiring.

I3 · done

Result remains speculative in the ROB.

I4 · done

Also waits until all older instructions are safe.



Memory Operations Need Ordering and Conflict Checks

01	Address generation	Compute effective addresses
02	Load queue	Track in-flight loads
03	Store queue	Track stores and forward matching data
04	D-cache	Serve data accesses
05	TLB	Translate virtual addresses
06	Ordering	Detect conflicts and preserve memory semantics

Part 7

Memory Hierarchy and System Architecture

The processor is useful only when instruction and data movement is fast enough and I/O is integrated correctly.

The Memory Wall

FAST CORE · SLOWER MAIN MEMORY

A small cache-miss rate multiplied by a large miss penalty can dominate CPI.



CPU core

Executes on short cycles and can demand several operands per cycle.



Caches

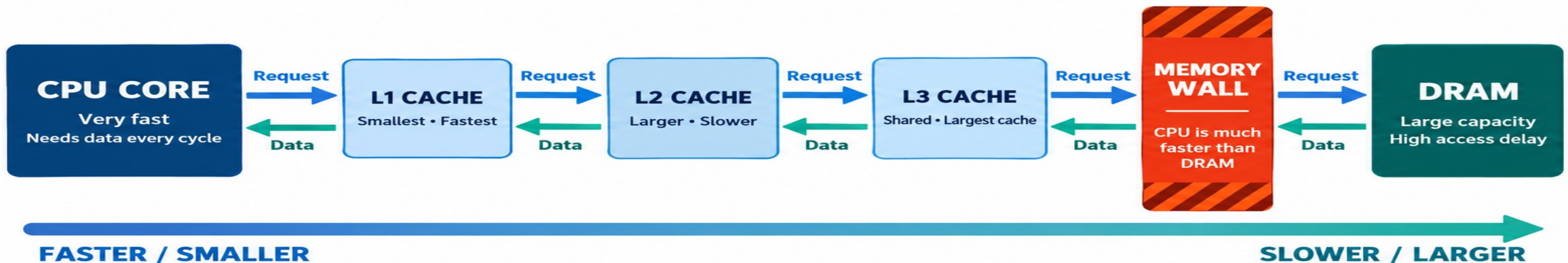
Keep frequently used instructions and data close to the core.



DRAM

Provides large capacity but at substantially higher latency.

Why Cache Is Needed: The Memory Wall



CACHE HIT

Data found in cache
CPU continues quickly



CACHE MISS

Data not in cache
Fetch from DRAM → CPU waits

Principle of Locality

Caches work because programs tend to reuse recent data and access nearby addresses.

Temporal locality

Recently accessed data is likely to be accessed again

Examples: loop variables and hot instructions

Supports keeping active blocks in cache

Reuse distance influences cache effectiveness

Spatial locality

Nearby addresses are likely to be accessed soon

Examples: arrays and sequential instructions

Supports fetching an entire cache line

Access patterns influence useful line size

Small and Fast Near the Core; Larger and Slower Farther Away

01	Registers	Fastest and smallest
02	L1 cache	Very low latency per core
03	L2 cache	Larger private or shared capacity
04	L3 cache	Large shared last-level cache
05	DRAM	Main memory
06	Storage	Largest capacity and highest latency

Cache Operation: Hit or Miss

TAGGED COPY OF MEMORY

A cache stores selected memory blocks plus tags that identify them.

CPU request

An address arrives from instruction fetch or a load/store.

Tag + index lookup

Choose the candidate set and compare stored tags.

Cache hit

Requested block is present — return data quickly.

Cache miss

Fetch the block from lower memory, install it, then return data.

Cache Address: Tag, Index, Offset

DECOMPOSE THE ADDRESS

Address fields identify the memory block, candidate set and byte within the block.



Tag

Which memory block is currently stored in the selected line or way?



Index

Which cache set or line should be searched?



Offset

Which byte or word within the cache block is requested?

Cache Policies

BEYOND PLACEMENT

Writes, replacement and prefetching are separate architectural choices.



Write policy

Write-through or write-back; write-allocate or no-write-allocate.



Replacement

LRU, FIFO, random, pseudo-LRU or RRIP choose the victim line.



Prefetch

Next-line, stride, software or hardware mechanisms bring data early.

Data Prefetcher

Predict future data addresses and fetch them before the load instruction arrives.

Observed stream

A

A + S

A + 2S

Predict A + 3S

Issue a prefetch before demand

Pattern detection

Identify streams, strides or correlations in recent accesses.

Benefit

Hide part of DRAM latency and increase memory-level parallelism.

Risk

Wrong predictions waste bandwidth and can pollute useful cache contents.

DRAM Access: Row then Column

BANKED MAIN MEMORY

Open-page and closed-page policies trade row-hit speed against predictability under conflicts.



1 · Row address

RAS activates the selected row within a DRAM bank.



2 · Row buffer

The entire row is sensed and held open.



3 · Column address

CAS selects the requested words from the active row.

TLB and Page Table Walker

VIRTUAL → PHYSICAL

Translate software virtual addresses into physical addresses used by cache and memory hardware.

Virtual address

Generated by instruction fetch or the load/store unit.

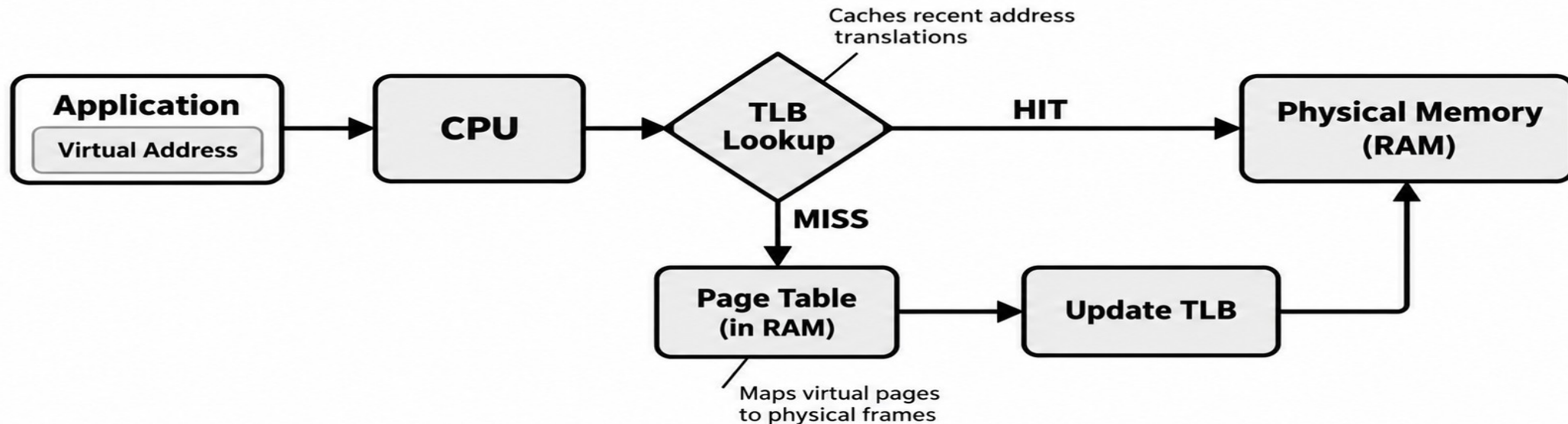
TLB

Caches recently used virtual-to-physical translations.

Page table walker

Consults page tables on a TLB miss and returns the physical mapping.

TLB and Page Table Walker



Why Virtual Memory?

Larger logical address space • Isolation and security • Simpler memory management

Latency vs Bandwidth

These are different dimensions of memory and I/O performance.

Latency

- Time from request to first useful data
- Critical for dependent loads and pointer chasing
- Cache miss penalty is a latency cost
- Lower latency shortens the critical path

Bandwidth

- Rate at which data can be transferred
- Critical for large blocks and streaming
- Parallel banks and channels improve throughput
- High bandwidth does not guarantee low latency

Memory-Mapped I/O (MMIO)

DEVICE REGISTERS IN THE ADDRESS SPACE

The CPU uses normal load/store instructions to access peripheral control and status registers.



Address space

RAM, ROM and device registers occupy defined address ranges.



Address decode

Hardware routes each access to memory or the selected peripheral.



Peripheral

UART, GPIO and timers expose control, status and data registers.

Polling vs Interrupts

Two ways for software to learn that a device needs attention.

Polling

- CPU repeatedly checks a status register

- Simple and low latency in tight loops

- Consumes CPU time while waiting

- Useful in simple single-purpose systems

Interrupts

- Device signals the CPU on an event

- CPU can do other work between events

- Requires ISR and interrupt-control logic

- Useful for multitasking and low-power systems

Direct Memory Access (DMA)

OFFLOAD BULK DATA MOVEMENT

The CPU sets up the transfer and handles completion; the DMA controller moves the data directly.



1 · CPU setup

Program source, destination, transfer count and direction.



2 · DMA transfer

Controller moves data between memory and the device.



3 · Completion

Interrupt or status indicates the transfer is finished.

Modern Processor: One-Picture Summary

KEEP THE MACHINE FED

Performance comes from supplying the front-end, scheduling the back-end and hiding memory latency.

Fetch + predict

Branch predictor, BTB, I-cache and prefetch deliver useful instructions.

Decode + rename

Translate instructions and map architectural to physical registers.

Issue + execute

Schedule ready work across ALU, FPU, SIMD and load/store units.

Retire + memory

ROB commits in order while caches, TLBs and DRAM serve data.

Use This Sequence to Reason About Any Processor or SoC

01	Digital logic	Gates + state
02	ISA	Programmable contract
03	Datapath + control	Execute instructions
04	Pipeline	Overlap stages
05	Modern core	Predict + schedule
06	Memory + I/O	Complete system integration